Improving Concurrent GC for Latency Critical Services in Multi-tenant Systems

Junxian Zhao
University of Colorado
Colorado Springs
CO, USA

Aidi Pi
University of Colorado
Colorado Springs
CO, USA

Xiaobo Zhou*
University of Colorado
Colorado Springs
CO, USA

Sang-Yoon Chang
University of Colorado
Colorado Springs
CO, USA

Chengzhong Xu
University of Macau
Macau, China

ABSTRACT

For resource utilization efficiency, latency critical (LC) services are commonly co-located with best-effort batch jobs in datacenter servers. Many LC services, such as Cassandra and HBase, run in Java Virtual Machine (JVM). We find that LC services often experience heavy-tailed latency due to performance interference of the concurrent garbage collection (GC) as well as multi-tenancy. The root cause is a semantic gap of resource allocation between JVM and the underlying Linux OS in multi-tenant systems. That is, the OS is unaware of the characteristics of different kinds of threads in JVM (i.e., GC threads and LC worker threads), which may lead to GC threads competing for CPUs; JVM is unaware of the resource utilization in the OS, which may trigger CPU-intensive GC operations when CPUs are busy. Furthermore, we find that co-located batch jobs can interfere with LC services due to Simultaneous Multi-Threading (SMT).

We propose iGC, a middleware that bridges the semantic gap between JVM and Linux OS and improves concurrent GC performance in multi-tenant systems. iGC adaptively triggers GC based on the CPU utilization at runtime, which speeds up the GC process and reduces its CPU contention. Furthermore, iGC deploys a dynamic CPU scheduling and thread placement strategy to avoid or mitigate the interference due to concurrent GC and multi-tenancy, but also improve the cache performance. We implement iGC upon two state-of-the-art concurrent GC mechanisms ZGC and G1 GC. We conduct its evaluation using three NoSQL databases as LC services. Experimental results show that iGC significantly improves the performance of concurrent GC for LC services and the throughput in multi-tenant systems. iGC reduces the p95 tail latency by 83%, 37% and 22% for the three LC services Cassandra, HBase and Solr, respectively. It also increases the throughput of LC services up to 2.56X.

*Corresponding author. Email: xzhou@uccs.edu.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Middleware '22, November 7–11, 2022, Quebec City, Quebec, Canada
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-8534-3/21/12…$15.00
https://doi.org/10.1145/3464298.3493394

CCS CONCEPTS

• **Computer systems organization** → **Cloud computing**; • **Software and its engineering** → **Garbage collection**.

KEYWORDS

Garbage collection, Job co-location, Tail latency, Interference

ACM Reference Format:
Junxian Zhao, Aidi Pi, Xiaobo Zhou, Sang-Yoon Chang, and Chengzhong Xu. 2022. Improving Concurrent GC for Latency Critical Services in Multi-tenant Systems. In *23nd International Middleware Conference (Middleware '22), November 7–11, 2022, Quebec City, Quebec, Canada.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3464298.3493394

1 INTRODUCTION

For resource efficiency in cloud computing, latency critical (LC) services such as key-value stores and web search are commonly co-located with best-effort batch jobs in datacenter servers. However, as applications are co-located and contend for shared resources, multi-tenancy often leads to severe interference and unpredictable performance, especially the heavy-tailed latency for LC services.

The heavy-tailed latency problem is significant because cloud applications are undergoing a major redesign from large, monolithic services to hundreds or thousands of loosely-coupled microservices [6, 25]. A user request of LC services often triggers sub-requests to multiple components across many servers and the end-to-end response time is largely determined by the slowest individual latency of the sub-requests [17]. As LC services need to maintain stable and low latency for user requests, the problem significantly impacts the QoS of cloud computing.

There are growing interests that aim to improve the tail latency of LC services from the perspectives of workload placement [55], CPU isolation between LC services and batch jobs [31], tail-aware caching [4], and resource partitioning [6, 36]. Recently, Caladan [24] proposes a new Linux OS scheduler that reduces the tail latency in multi-tenant systems via fast core allocation instead of resource partitioning. Hermes [46] and Perséphone [20] design reservation-based memory allocation and kernel-bypass OS scheduling approaches to minimize the tail latency for LC services executing at the microsecond scale, respectively.

In this paper, we tackle the problem of heavy-tailed latency of LC services from two new aspects. First, we identify a new interference causing the heavy-tailed latency in JVM-based LC

services, that is, the interference between the concurrent GC threads and the worker threads of a LC service. Many LC services, such as Cassandra [1], HBase [2], and Solr [3], run in Java Virtual Machine (JVM). JVM provides an easy-scalable, generalized platform for different workloads, which fits well in cloud environments. While Java provides programmers with convenience such as memory management, scalability, and compatibility, it also incurs nontrivial overhead of GC. Traditional GC in JVM leads to the Stop The World (STW) problem, which pauses all worker threads [13]. Concurrent GC, represented by ZGC [16] (since JDK 11) and G1 GC [22] (since JDK 7u4, the default in JDK 9), runs most phases of GC concurrently with worker threads of Java applications. It reduces the long pause associated with STW and the full GC. However, it creates severe interfere with the worker threads of LC services.

The root cause is a semantic gap of resource allocation between JVM and the Linux OS in multi-tenant systems. 1) JVM is unaware of the available resources in the OS. It triggers GC only based on the status of JVM, which often prolongs the execution time of both GC threads and threads of batch jobs that share CPUs with the GC threads. 2) The OS is unaware of the characteristics of different threads in JVM, which may lead to threads competing for the same set of logical CPUs. In such a case, GC threads contend for CPU resource with worker threads of LC services in JVM, which prolongs the latency of requests of LC services.

Second, servers enable Simultaneous Multi-Threading (SMT) technology to improve system throughput. Linux OS may schedule different workloads on the logical CPUs running on one physical core by SMT technology. As a result, co-located batch jobs can incur non-trivial SMT interference with LC services, leading to cache contention and longer request latency of LC services.

We propose iGC, a middleware that bridges the semantic gap between JVM and the OS and improves concurrent GC performance in multi-tenant systems. 1) iGC adaptively triggers GC based on the available CPU resource at runtime, which speeds up GC process and reduces its CPU contention. 2) iGC deploys a dynamic CPU scheduling and thread placement strategy to avoid or mitigate the interference due to concurrent GC and multi-tenancy, but also improve the cache performance at the same time.

Specifically, iGC separates GC threads from worker threads of LC services by assigning different CPUs for the GC threads and worker threads, avoiding the interference. To mitigate SMT interference, iGC tries to assign threads of batch jobs and worker threads of LC services to different cores. Further, iGC limits the resource usage of batch threads when GC threads are co-located with batch threads and GC is triggered. As a result, it effectively mitigates the contention of memory hierarchy in a multi-tenant system.

We implement iGC as a middleware upon two state-of-the-art concurrent GC mechanisms, ZGC and G1 GC. We conduct performance evaluation using three representative NoSQL databases Cassandra, HBase, and Solr as the LC services and Spark KMeans as the co-located batch jobs. Experimental results show that compared to ZGC, iGC reduces the p95 latency by 83%, 37% and 22% for Cassandra, HBase, and Solr, respectively. It reduces the L1 and L2 cache miss rate by 55.7% and 54.4%, respectively. It also increases the instruction throughput of the LC services up to 2.56X. Compared to G1 GC, iGC reduces the p95 latency by 78% and improves the instruction throughput up to 1.45X for Cassandra.

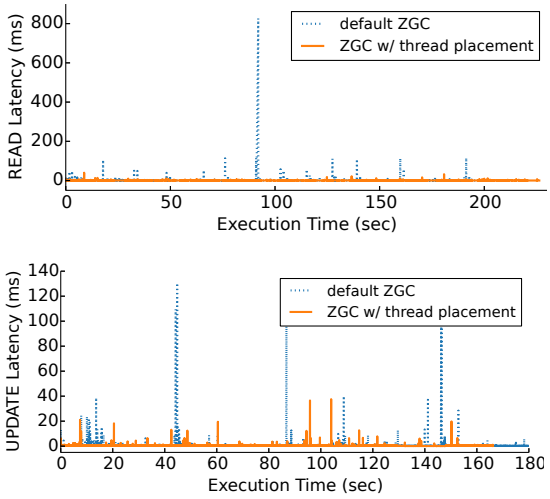


Figure 1: Latency of Cassandra operations at runtime.

The rest of this paper is organized as follows. Section 2 introduces the background and motivational studies. Sections 3 and 4 describe iGC design and implementation, respectively. Section 5 presents the testbed setup, experimental results and analysis. Related work is presented in Section 6. We conclude the work in Section 7.

2 BACKGROUND AND MOTIVATIONS

2.1 Concurrent GC Interference

Though concurrent GC is effective in mitigating the STW overhead, GC threads compete CPU resources with other threads which can severely increase the latency of LC services and reduce the system throughput. In a multi-tenant system, GC threads compete with threads of multiple types of workloads. 1) Different threads within one JVM compete for CPUs, that is, concurrent GC threads compete with worker threads, which prolongs the request latency of the LC service. 2) Threads of batch jobs (batch threads) in one JVM share and compete for CPUs with GC threads of a LC service in another JVM, which prolongs the GC process.

To study the impact of concurrent GC on latency of LC services, we run Cassandra, a NoSQL database, as the LC service on Java 11 with ZGC. We use YCSB [12] as the benchmark to send requests to the LC service. The experimental setup is same as that in Section 5.1. We compare the default ZGC and ZGC with adaptive thread placement. For the default ZGC, GC threads may be scheduled on the same set of CPUs that host worker threads of Cassandra. For the ZGC with adaptive thread placement, GC threads are assigned to CPUs that do not host worker threads of Cassandra.

Figure 1(a) shows the latency of the Read and Update operations of Cassandra at runtime. Compared to ZGC with adaptive thread placement, the default ZGC leads to more frequent GC activities and longer GC time. The reason is that worker threads and GC threads share the same set of CPUs, leading to severe contention. The longest latencies are due to the ZGC stalls when there is not enough memory on the JVM heap for allocation before the GC is finished. Separating ZGC threads and placing them onto CPUs that

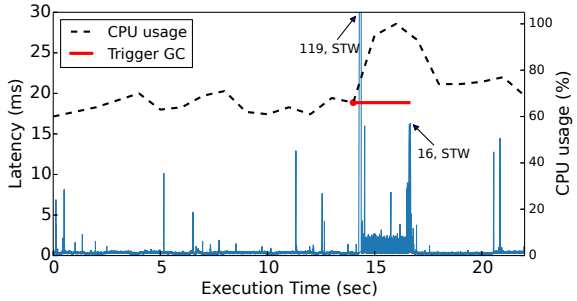


Figure 2: Correlation of GC, CPU utilization, and tail latency.

do not host worker threads of Cassandra can render smaller and more stable latency. By placing GC threads and worker threads onto different CPUs, GC runs faster without CPU contention which avoids the ZGC allocation stall. As Cassandra employs the log-structured merge-tree, the latency of Update operations is relatively more fluctuating due to write amplification in which one write operation triggers multiple times of physical writes to the disk.

We further illustrate the correlation of GC triggering with the CPU utilization and latency of Cassandra. We use YCSB workload A that consists of 50% of Read and 50% of Update operations. We set the number of requests to 4,000 operations per second. Figure 2 shows the request latency and CPU utilization at runtime. The average request latency during the runtime is about 0.35ms. A full GC is triggered around the 14th second and lasts about 2.4 seconds (the red line). During the GC process, the CPU usage spikes to nearly 100%, more than 30% higher than the usual. Meanwhile, the latency also increases significantly. There are two peaks of latency occurring after the GC start point and before the GC end point due to STWs. The first peak of latency is around 119ms. After the first STW, it is the concurrent marking phase of GC. During this period, the average latency jumps to around 3ms, which is about 9x larger than the average latency. The second STW is less severe, causing the request latency to 16ms. Overall, though concurrent GC reduces the STW overhead, the CPU interference during the concurrent marking phase leads to severe degradation of LC service latency.

2.2 SMT/Hyper-Threading Interference

SMT has been widely enabled in modern multi-core processors. It improves the system throughput by scheduling instructions from multiple hardware threads (each is called a logical CPU) in one physical core, which can concurrently utilize different execution units on the core. In this paper, we target Intel's implementation of SMT, which is named Hyper-Threading (HT). We use CPU to represent logical CPU and core to represent physical core. HT implements a 2-way hardware threads on a physical core. Two CPUs on a core share the same pipeline and L1, L2 caches.

While HT technique improves the resource efficiency, it also incurs non-trivial overhead to the both hardware threads on the core for two reasons. 1) The core is unaware of the hardware threads from which instructions come. There is not a mechanism that can prioritize a thread with a better opportunity for occupying the pipeline. When two threads run on the two CPUs of the same core, each thread executes instructions slower compared to that one

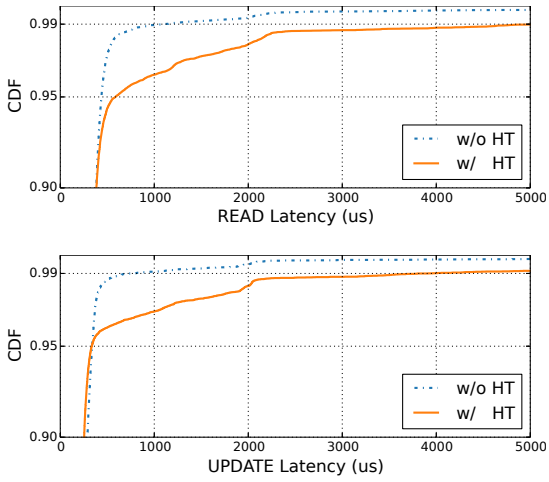


Figure 3: The latency CDF of all requests (μs) for Cassandra.

thread runs exclusively on the core. The contention of execution units on the core delays the execution of both threads. 2) Contention also occurs in memory hierarchy. Two CPUs of the core share resources, especially the memory hierarchy, i.e., L1, L2 caches. As more cache misses lead to more accesses to the next level of the memory hierarchy, the execution of instructions is prolonged which increases the execution time of batch jobs or LC services [29].

We use a case study to show the impact of HT on latency of LC services in two scenarios. HT is enabled: worker threads of Cassandra and batch threads of KMeans run on two CPUs of the same core, respectively. HT is disabled: worker threads of Cassandra run on one CPU while the other CPU of the core is idle. Figure 3 shows the latency of Read and Update operations of Cassandra with HT enabled and disabled. We observe that about 5% of operations are significantly delayed when HT is enabled. The latency impact on LC services is due to the two identified causes, i.e., contention of the execution units and contention in the memory hierarchy.

Summary. We summarize the identified issues as below. 1) Garbage Collection is a CPU intense operation. Thus, it should be adaptively triggered based on the CPU resource at runtime. 2) Although concurrent GC reduces the stall of applications, it leads to more CPU interference as it runs in parallel with worker threads of the same process. 3) Assigning GC threads and batch threads on the CPUs of the same core severely degrades the tail latency of LC services due to CPU contention and HT interference.

3 IGC DESIGN

We design iGC, a middleware that improves concurrent GC performance in multi-tenant systems and reduces the latency of LC services by addressing the identified issues. iGC centers on two key techniques. 1) Adaptive GC triggering. iGC triggers GC adaptively based on the available CPU resource at runtime to speeds up GC procedure and reduce its CPU contention. It monitors the CPU usage of servers and dynamically adjusts the time interval between GC operations based on the heap usage of the JVM. 2) Dynamic CPU allocation and thread placement. iGC deploys a simple yet

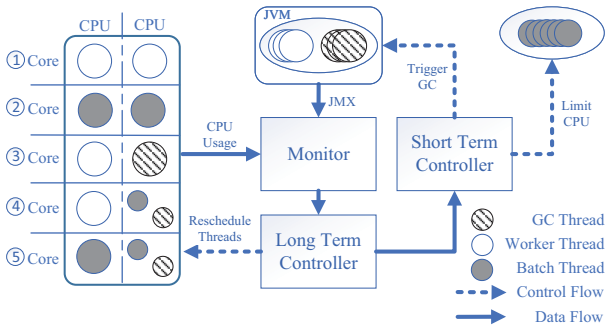


Figure 4: The architecture of iGC with five CPU and thread placement scenarios.

effective priority-based dynamic CPU allocation and thread placement strategy to reduce the interference of job co-location and improve the cache performance by leveraging HT.

Figure 4 shows the architecture of iGC. iGC consists of three units: 1) Monitor that obtains server resource information, i.e., heap size of the target application, CPU utilization for each core on the server. It uses samples to calculate the speed at which the heap size grows. 2) Short term controller that triggers GC based on the information from the other two units. It also limits the resources used by the batch threads when necessary. 3) Long term controller that decides the parameters related to GC triggering, such as thresholds of heap size, CPU utilization and time interval of GC, and importantly CPU allocation and thread placement. A JVM is the runtime for a LC service. It contains worker threads and GC threads. In addition, there are batch threads for batch jobs. There are five thread placement scenarios on the two CPUs of a core, i.e., worker and worker, batch and batch, worker and GC, worker and batch with GC, and batch and batch with GC. In case of virtualized cloud environments, each virtual machine (VM) hosts one iGC that makes GC triggering and CPU scheduling decisions for the hosted applications. Each VM can access multiple cores on the server. Note that scenario 3 only occurs when resources are plenty, otherwise, it will be transformed to scenario 4 or 5 for higher resource utilization.

3.1 Monitor

The monitor periodically collects the resource utilization information including CPU usage and heap usage of a target JVM. The CPU usage is obtained from the file system in Linux OS. The heap usage of the JVM is obtained through the Java Management Extensions (JMX) framework introduced since Java 1.5 [15]. JMX is an out-of-the-box management tool for monitoring and managing JVM. The monitor also tracks the allocation of objects on the Java heap and computes the allocation rate. The obtained information is sent to the short-term controller and the long-term controller.

3.2 Short Term Controller

The controller manages the adaptive GC triggering procedure. The logic is shown in Algorithm 1. It triggers GC based on CPU usage, heap usage, and interval set by Monitor and the long term controller. It limits the CPU usage of batch threads on the allocated CPU where GC threads are running (scenarios 4 and 5 in Figure 4). One naive way is to remove the allocated CPU from the affinity list of batch threads. Here, we choose to set low priority in CPU usage for batch threads so they can be resumed when GC is finished.

3.2.1 How to trigger GC. iGC decides how to trigger GC at runtime. Instead of modifying the JVM directly, we access JVM information and control GC via JMX. JMX obtains the status of Java heap and triggers GC via call `system.gc()` from outside of the JVM. Algorithm 1 shows the operations when GC is triggered. By default, GC threads may share a CPU with batch threads. If the JVM is started with root privilege, GC is triggered directly as the OS scheduler will treat it with high priority. If not, GC is triggered if *cpu_usage* of the CPU where GC threads run is smaller than *cpu_threshold*, which is empirically set to 70% to reduce the CPU contention between GC and batch threads. Note that with iGC, the GC CPU usage on each assigned CPU is around 30%. To speed up GC operations, we set GC threads with a high priority on the CPUs where GC is triggered. If the CPU resource is not enough, iGC limits the CPU usage of batch threads sharing the same CPU with GC threads by cgroup [11].

Algorithm 1 The procedure of triggering GC.

```
1: cpu_threshold: the CPU usage threshold;
2: gc_threads: the list of GC threads;
3: batch_threads: the list of batch threads sharing CPU with GC threads;
4: modify_batch: a flag to set low priority for batch threads;
5: function TRIGGER_GC(cpu_usage)
6:     if ISRUNNINGWITHROOTPRIVILEGE() then
7:         DO GC()
8:         return
9:     end if
10:    if cpu_usage < cpu_threshold then
11:        LIMITCPUUSAGE(batch_threads);
12:        modify_batch = 1; //set low priority to batch threads
13:    end if
14:    PRIORITIZETHREADS(gc_threads);
15:    DO GC()
16:    if modify_batch == 1 then
17:        RESUMECPUUSAGE(batch_threads);
18:    end if
19:    DEPRIORITIZETHREADS(gc_threads);
20:    return
21: end function
```

3.2.2 When to trigger GC. There are two conditions for the timing of triggering GC, 1) There is enough CPU resource so it proactively triggers GC, and 2) the heap usage increases beyond a threshold such that GC must be triggered to avoid the allocation stall. iGC keeps monitoring the heap usage of the JVM and the CPU usage of the server. It periodically checks whether GC should be triggered by a dynamic time interval I and two thresholds. Threshold A is a threshold for the heap usage, below which GC is not triggered as there is enough heap space for allocation. iGC tries to trigger GC when *heap_usage* is larger than threshold A. In this case, if *heap_usage* is not larger than threshold B, it triggers GC if *cpu_usage* is smaller than *cpu_threshold*. If *heap_usage* is larger than threshold B, GC is triggered regardless of the value of *cpu_usage*. Using two thresholds reduces the interference to batch threads. To ensure iGC is properly working, the default concurrent GC is disabled. Note when the heap size reaches a very high level between iGC samplings, the JVM can still trigger GC to avoid the stall of LC services.

Table 1: A typical thread list from the JVM of Cassandra.

	Worker Thread & JVM Priority	GC Thread
Label	(5)RMI TCP Accept (5)MemtableFlushWriter (5)MemtableReclaimMemory (5)SecondaryIndexManagement (5)ScheduledTasks (5)NonPeriodicTasks (5)logback#1~n (5)RuntimeWorker#0~n (9)Signal Dispatcher (9)Attach Listener (9)C1CompilerThread (9)C2CompilerThread (9)Service Thread (9)VM Thread (9)VM Periodic Task Thread (10) Reference Handler (10) Process Reaper (10) epollEventLoopGroup	ZGC: ZDriector ZDriver ZStat ZWorker#0~n Sweeper thread G1GC: GCThread#0~n G1 Conc#0~n G1 Refine#0~n G1 RemSet Sampling
JVM Priority	5 ~10	10
OS Priority	0	0

3.3 Long Term Controller

This unit has two key functions, dynamic CPU allocation and thread placement and adaptive monitoring interval. The former addresses non-trivial CPU contentions between GC threads and worker threads and between batch threads and worker threads. It also leverages HT for cache performance improvement. The latter controls parameters used in GC, including the heap usage threshold, the CPU usage threshold, and the number of GC threads.

3.3.1 Priority-based Dynamic Thread Placement. There is not a thread scheduler in JVM itself. A JVM sets different priority levels to Java threads and creates an OS-level thread for each Java thread. The Linux OS schedules Java threads based on their priority.

JVM Thread Priority. All GC threads are initialized when a JVM starts runtime. We use command `jps` to get process ID and command `jstack` to capture the thread dump of the JVM and obtain thread labels and IDs. Table 1 shows the label and priority level of all GC threads and representative worker threads in the JVM of Cassandra. Note that ZGC and G1 GC use different labels for GC threads. We can see that most of the worker threads have JVM priority levels from 5 to 9, except for the last three threads that have the highest priority level 10. All GC threads have the highest JVM priority level 10. The last row of the table shows that all JVM threads are assigned the priority level 0 by the OS when the JVM is initiated at user space. As a result, for the JVM running in user space, GC threads and worker threads are treated equally by the Linux OS scheduler and the use of round-robin time-slicing based scheduling. When the JVM is initiated with root privilege, the priority levels of the threads are converted to the OS-level priority levels by the JVM and the OS schedules the threads based on their priority.

Under both scenarios, the OS is incapable of avoiding or mitigating the interference between the GC threads and worker threads by its scheduler itself. If GC threads are assigned with a higher priority level than those of worker threads, worker threads will be preempted or reduced time slice size when GC is triggered, which prolongs the request latency. If GC threads are assigned with a lower priority, it may lead to expensive STW problems or even `OutOfMemory` errors.

iGC Dynamic Thread Placement Strategy. To avoid or mitigate the interference due to concurrent GC and job co-location but also increase the cache performance, iGC places three types of threads on each core with five scenarios in order, which are illustrated at the left side of Figure 4. 1) It assigns both CPUs of one core to worker threads, which avoids the interference from batch threads and makes use of the memory hierarchy of the core. 2) It assigns both CPUs of one core to batch threads, which avoids the impact on worker threads. 3) When there are enough CPUs, worker threads and GC threads are assigned to the two CPUs of one core, respectively. 4) When CPUs are not enough, GC threads share the CPU with batch threads on one CPU, while worker threads use the other CPU. 5) In the worst scenario, GC threads share one CPU with batch threads, while other batch threads use the other CPU of the same core.

The CPU and thread placement is dynamic. Firstly, iGC places worker threads and batch threads into different cores (scenarios 1 and 2). If the CPU usage of the LC service is higher than 90% during a sampling interval, one more core is assigned to the LC service. If there are more batch jobs and the CPU usage is heavy on batch threads, iGC co-locates GC threads with batch threads on one CPU (scenarios 4 and 5) by orchestrating with the Linux OS scheduler.

We discuss three possible co-location scenarios at the CPU level: 1) Worker threads and GC threads, 2) batch threads and GC threads, and 3) Worker threads and batch threads.

Worker & GC Worker threads may compete with GC threads when they are placed on the same CPU. However, Linux OS does not distinguish the two types of threads at runtime. To address the problem, we collect information of Java threads from the JVM as each thread has its own label. Each Java thread in the JVM has its corresponding system thread in Linux. Those system threads share one process identification (PID). With the obtained labels, iGC distinguishes the two types of threads and pins them onto different CPUs, which avoids the contention between the worker threads and GC threads.

Batch & GC Though batch threads compete resources with GC threads, GC is not in the critical path of a LC service as the interference on GC threads from batch threads will not impact the latency of the LC service. GC threads are only active for a short interval. As long as it finishes before the heap size reaches the limit, there is not allocation stall. As a result, GC threads can share CPU resources with batch threads. By starting JVMs with root privilege, iGC co-locates GC threads with low-priority batch threads on one CPU for high utilization. It is the Linux OS scheduler that schedules the threads based on their priority. Alternatively, if JVMs are started at user space, iGC prioritizes GC threads by the use of `nice` command [27] to reduce the interference between batch threads and GC threads.

Worker & Batch Batch threads compete CPU and memory hierarchy resources with worker threads. Batch threads should not be placed in CPUs that execute worker threads.

3.3.2 Making use of HT. Two factors are considered when making use of the HT technique, the impact on the memory hierarchy and on the instruction throughput. Sharing the same physical core leads to contention on the core-level caches, i.e., L1 data cache, L1 code cache, and L2 cache. Each process owns one virtual memory space.

The data stored in caches can be reused by threads from the same process. An efficient way that takes advantage of the HT technique is to place worker threads on one CPU and GC threads on the other CPU of the same physical core. When worker threads and GC threads of the LC service share two different CPUs on the same core, they can share the cached data. It reduces the access to the lower levels of the memory hierarchy, i.e., LLC cache and the main memory. In return, the fast data fetching reduces the stall in the instruction pipeline and increases the instruction throughput as well.

3.3.3 Adaptive monitoring interval. iGC uses an adaptive monitoring interval (I) that is initially set to 5 seconds, and then dynamically changed at runtime according to the heap allocation growth. It aims to reduce the possibility of triggering the default GC while reducing the monitoring overhead. Algorithm 2 shows the procedure of the adaptive monitoring interval. The interval of obtaining the resource usage and heap usage varies in different configurations. The frequency of GC is also dynamically changed based on the allocation rate of new objects in the Java heap. The allocation rate is calculated by the number of new objects allocated in Java heap per second. iGC keeps sampling the allocation rate of new objects and computes the new value of the interval.

In Algorithm 2, parameter u is the coefficient that determines the weight of the new rate and the current rate in the new monitoring interval. The formula at line 6 is to get the new interval for triggering GC. The new interval is calculated based on the current interval, the current allocation rate, and the new allocation rate. The allocation rate is volatile in Java heap. Since it changes frequently, we choose to give a higher weight for the new allocation rate. To reduce the fluctuation from bursty heap allocations, the value of u is empirically assigned to 0.2. As a result, the interval of triggering GC can adaptively change at runtime.

Algorithm 2 Adaptive monitoring interval.

```
1:  I: the time interval of triggering GC;
2:  cur_rate: the current allocation rate of new objects;
3:  u: the coefficient (0<u<1);
4:  function Test_GC
5:      new_rate = getCurrentAllocationRate()
6:      I = u*I + (1-u)*I*(new_rate - cur_rate)/cur_rate;
7:      adjustGcThreadNumber(cur_rate, new_rate);
8:      adjustHeapUsageThreshold(cur_rate, new_rate);
9:      cur_rate = new_rate;
10:     return (n)
11: end function
```

4 IMPLEMENTATION

iGC, implemented as a lightweight middleware between JVM and Linux OS, is written in Bash and C++ with around 1,000 lines of code. The implementation tackles several challenges. Firstly, its monitor needs to collect information from the JVM, i.e., different thread types, heap usage, object allocation rate, but also from the Linux OS, i.e., CPU usage for each core. Secondly, iGC needs a suitable interface for controlling JVM parameters at runtime. Last but not least, iGC needs to be compatible with the two state-of-the-art concurrent GC mechanisms ZGC and G1GC. iGC project is open source at https://github.com/lotusknight/iGC.

iGC daemon process runs with the root privilege to access the information of resource usages and control resource scheduling on servers. It obtains CPU usage information from the Linux file `/proc/stat`. It samples the CPU usage in each second.

JMX facilitates programmers to monitor and manage JVM. iGC uses JMX to access the Java heap usage and trigger GC from outside of the JVM. With JMX, iGC can be deployed for all Java applications without modifying the source code of applications. When employing JMX, we assign JMX port number for each Java application when starting the JVM.

iGC has interfaces to ZGC [16] and G1 GC [22]. Note that iGC and JVM won't trigger GC threads at the same time as iGC is a middleware that triggers GC through JVM GC interface. Once iGC has triggered GC, it will change a status flag in JVM so that no more GC will be triggered at the same.

iGC uses scripts to obtain the PIDs of the JVMs for LC services and batch jobs. It uses command `jstack` to capture the JVM thread dump and obtain thread labels and thread IDs. Threads both related to GC and worker are recognized as worker threads. With the labels and thread IDs, iGC distinguishes the GC threads and places them onto CPUs according to the priority-based dynamic thread placement strategy. iGC sets the thread CPU affinity by using `taskset` command. We use Linux cgroup to adjust the CPU limits for GC threads and batch threads which share the same CPUs. iGC can adapt to JVMs in both user space and root privilege. If JVMs are started at user space, command `nice()` is used to set GC threads with higher priority when they are co-located with batch threads to reduce the CPU interference. If JVMs are started with root privilege, iGC dynamic thread placement avoids or mitigates the interference.

5 EVALUATION

5.1 Evaluation Setup

We use two servers in the evaluation. Each server has two Intel Xeon E5-2630 v3 @ 2.40GHz CPUs 16 cores (32 hyper threads in total, L1 cache 320KB (data), 320KB (code), L2 cache 2.5MB, Haswell architecture with eight execution units), 32 GB DRAM, and 1TB HDD.

LC services. We use three NoSQL databases as LC services, Cassandra-4.0-beta2, HBase-2.2.7, and Solr-8.8.2. Cassandra, a wide-column store, is a popular NoSQL database management system that is designed to handle large amount of data across servers. HBase runs on top of HDFS [48], providing Google Bigtable-like capabilitiy for Hadoop. Solr is a specialized database designed for read only (scan is a sequential of reads). They are commonly used as LC services for performance evaluation. Three databases are running with JDK 11. The heap size of each database is set at 8GB.

We use YCSB [12] as the benchmark to send requests to the LC services. YCSB is an open-source specification and program suite, which is specifically used to compare relative performance of NoSQL database management systems. It offers workloads containing different operations including read, update, insert, and scan. Table 2 shows five types of workloads and the percentage of operations in the each workload used in evaluation. As Solr is mainly used as a search engine, we only use read and scan workloads (Workloads C and S) for evaluation. Unless mentioned, a YCSB Client sends operations in best effort per second in our experiments.

Table 2: The Percentage of Operations in Workloads.

Operation	READ	UPDATE	INSERT	SCAN
Workload A	50%	50%	0	0
Workload B	95%	5%	0	0
Workload C	100%	0	0	0
Workload D	95%	0	5%	0
Workload S	0	0	0	100%

Batch jobs. We use Yarn in Hadoop-2.9.2 [49] as the scheduler for batch jobs. Spark-2.1.0 runs KMeans workload from HiBench-6.0 [30] as the batch jobs. Batch jobs runs with JDK 8. One server works as the master of Yarn and the client of LC services. The other serves as the multi-tenant system where LC services and best-effort batch jobs are co-located, where iGC is installed. All experiments were conducted under co-location, unless those marked Alone.

Approaches. We primarily use ZGC as the concurrent GC in iGC performance evaluation. ZGC is the latest concurrent GC, which is of low latency. We also use the other popular concurrent GC, G1GC, in the evaluation.

Metrics. We run three NoSQL databases with different workloads to evaluate the latency of LC requests and the throughput of LC service and batch jobs. Then, we use cache miss and the number of retired instructions as metrics to illustrate the benefit of taking advantage of memory hierarchy in iGC. We then test the performance of iGC under different pressure levels. Next, we present the latency of each operation in Cassandra. Finally, we discuss the overhead of iGC.

5.2 Latency and Throughput

We evaluate the latency of LC services as well as the throughput of both LC services and batch jobs using the three NoSQL databases. Experiments are conducted in three configurations: 1) LC services running alone using ZGC (Alone w/ ZGC), 2) LC services co-located with batch jobs using ZGC (Co /w ZGC), and 3) LC services co-located with batch jobs using iGC (Co w/ iGC). Under each configuration, we run the experiment for one hour and measure the average values.

5.2.1 Latency Reduction. The tail latency CDF of all requests of LC services Cassandra, HBase and Solr are shown in Figure 5, Figure 6 and Figure 7, respectively. Figure 5 shows that under all three configurations, Alone with ZGC shows the best performance for all Cassandra workloads as there is no interference. Compared to the co-location with ZGC, co-location with iGC shows significantly reduction of the tail latency in the all three workloads, especially for workload A. The reason is that iGC places GC threads and worker threads in different CPUs, which mitigates the interference of concurrent GC on worker threads of LC services.

Under co-location, most of the latency due to iGC and ZGC are concentrated around three values: 500μs (low latency), 4000μs (medium latency), and 8000μs (high latency), which form a stair-like shape. The reason is that there is a mechanism in Cassandra that maintains cache layers to speed up requests. iGC reduces the access to the next layer as it incurs less contention to memory hierarchy. As a result, iGC renders 6% (p96-p90) more requests that have low latency and 2% (p98-p96) more requests that have medium latency for workload A, in compared with ZGC.

Table 3: The Number of Finished Batch Jobs.

	Cassandra	HBase	Solr
Co w/ ZGC	189	187	189
Co w/ iGC	185	182	184
Standalone	203		

Figure 6 shows the experimental results for HBase workloads. iGC renders the lowest latency for requests almost at all percentiles. In HBase, CPU contention is very severe. By placing GC threads and worker threads in different CPUs, iGC is able to mitigate the interference while Alone with ZGC does not. Note that the interference from batch jobs results in longer p99 latency under iGC in comparison with Alone with ZGC. Importantly, iGC significantly outperforms co-location with ZGC.

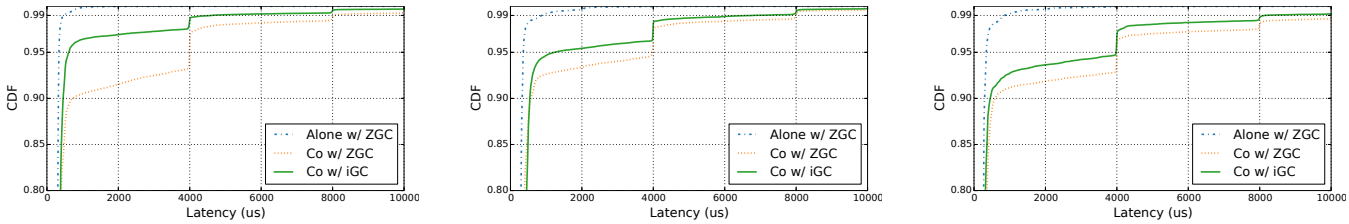
Figure 7 shows the experimental results for Solr workloads. iGC renders the best performance under workload C. The alleviation of CPU interference on worker threads leads to smaller latency for most requests. The SCAN operation sequentially reads data, which leads to higher cache miss rate. As a result, iGC renders results similar to that due to Alone with ZGC in workload S as it loses the advantage of memory hierarchy.

Figure 8 shows the reduction of the average latency and latency at four specific percentiles by iGC in comparison with ZGC for the three LC services under co-location. Compared with the default ZGC, iGC can reduce the p95 latency by 83% (Workload A), 37% (Workload A), and 22% (Workload S) for Cassandra, HBase and Solr, respectively.

5.2.2 System Throughput. We measure the throughput of LC services in terms of the number of operations per second. Figure 9 shows the throughput for each LC service with different workloads by iGC and ZGC under co-location. iGC shows non-trivial throughput improvement for all eight workloads, because the separation of GC threads and worker threads significantly reduces the CPU interference on worker threads. Further, it limits the resource usage of batch threads when GC threads are co-located with batch threads and GC is triggered.

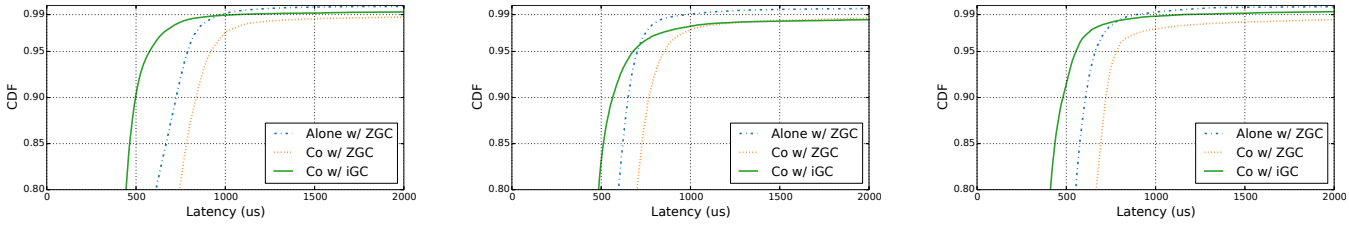
To measure the throughput of batch jobs, we count the number of the finished batch jobs. Table 3 shows the number of the finished batch jobs in one hour due to iGC and ZGC. As the baseline, the Standalone entry shows the number of batch jobs running alone on the server without the resource limit. The throughput of batch jobs due to iGC under co-location with Cassandra, HBase and Solr is only reduced by 2.2%, 2.6% and 2.7%, respectively. The reduction is due to two reasons. First, iGC separates GC threads and worker threads of LC services onto different CPUs, which uses more CPU resource in exchange for lower latency by mitigating the CPU resource contention. Second, iGC dynamically triggers GC with an adaptive time interval, which causes additional CPU resource but reduces the interference of GC on batch jobs. The impact of iGC on the throughput of batch jobs is trivial, because the STW time interval is small in concurrent GC. Under job co-location, the latency and throughput of LC services are the dominant performance metrics.

Due to the space limit, we use Cassandra as the representative LC service in the following evaluation.



(a) Workload A (b) Workload B (c) Workload D

Figure 5: The tail latency CDF of all requests of three Cassandra workloads.



(a) Workload A (b) Workload B (c) Workload D

Figure 6: The tail latency CDF of all requests of three HBase workloads.

Table 4: The Number of Each Monitor Events in One Minute.

Monitor Event	Description	Event No.	Co w/ ZGC	Co w/ iGC
L1D_PEND_MISS.PENDING	L1D misses every cycle.	0148h	$130 * 10^9$	$57.4 * 10^9$
L2_RQSTS.MISS	Requests that missed L2.	3F24h	$154 * 10^9$	$70.0 * 10^9$
L1D.REPALCEMENT	Lines evicted by new data lines.	0151h	$129 * 10^9$	$51.5 * 10^9$
INST_RETIRED.ANY_P	Instructions at retirement.	00C0h	$44.2 * 10^9$	$114 * 10^9$
UOPS_RETIRED.ALL	Micro-ops retired.	01C2h	$59.6 * 10^9$	$130 * 10^9$
UOPS_RETIRED.RETIRE_SLOTS	Retired slots used in each cycle.	02C2h	$48.8 * 10^9$	$109 * 10^9$

5.3 HT Impact on Cache Performance

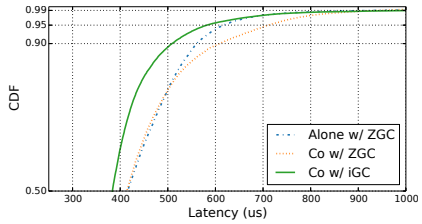
We use two dimensions (the cache miss and the instruction throughput), three cache metrics, and three types of retired instructions, to evaluate the impact of HT on cache performance under co-location with iGC and ZGC. We use Cassandra workload A in the evaluation. We obtain the counter values from the hardware monitor events of the Intel processors. We compare the number of cache misses and the number of retired instructions on a CPU where worker threads run. Table 4 gives six monitoring events related to cache performance [14].

The experiment runs under two different configurations: 1) Worker threads run on one CPU while GC threads run on the other CPU of the same core (co-location w/ iGC), and 2) Worker threads run on one CPU while batch threads run on the other CPU of the same core (co-location w/ ZGC).

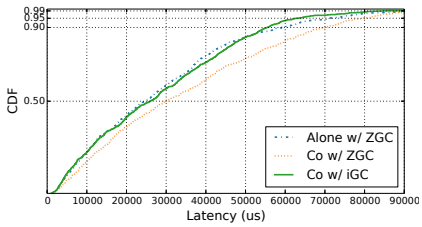
5.3.1 Cache Miss and Replacement. In Table 4, the first three entries show the cache miss of the Cassandra workload due to iGC. We count the operations for one minute for each event. We chose a 1-minute time interval as it is long enough to include 10 cycles of GC for analysis. Three metrics are 1) the number of L1 data cache misses per cycle, 2) the number of requests that missed L2 cache, and 3) the number of cache lines in L1 cache which are evicted by bringing in new line to L1 cache. Compared with the co-location with ZGC, iGC reduces the three metrics by 55.7%, 54.4%, and 59.9%, respectively.

With HT enabled, GC threads and worker threads are assigned to the two CPUs on the same core. Each core shares the same L1 cache and L2 cache. With iGC, the L1 cache miss and L2 cache miss decrease dramatically. We also use the L1 replacement as the metric. It shows that fewer cache lines are replaced by new cache lines in L1 cache. Compared with the co-location with ZGC, iGC shows less contention in memory hierarchy. Threads of LC services on both CPUs of the same core are from the same JVM. They share the same memory space such as Java heap, so the cache data can be reused by both threads and the cache misses are reduced. Additionally, it reduces the replacement of cache lines by batch jobs threads, which also reduces the cache miss.

5.3.2 Retired Instructions. The last three entries in Table 4 give the number of retired CPU instructions for LC services due to iGC. We count the operations for one minute for each event. Three metrics are 1) the number of all retired instructions, 2) the number of retired micro-operations (uops), and 3) the number of retired slots (a.k.a.



(a) Workload C



(b) Workload S

Figure 7: The tail latency CDF of all requests of two Solr workloads.

execution units) used in each clock cycle. There are eight slots in Haswell architecture that can be used in each cycle, which means eight micro-operations or eight instructions could be retired in each cycle. Compared with ZGC, iGC improves the three metrics by 2.56X, 2.18X, and 2.23X, respectively.

Each instruction consists of several micro-operations. An instruction is retired when it is finished on the CPU pipeline. Since iGC separates GC threads and worker threads onto different CPUs, the interference between GC threads and worker threads is avoided. As a result, the utilization of the retired slots per cycle is higher, so are the number of retired instructions and uops. iGC significantly increases the throughput of LC services.

5.4 Stress Test and Scalability

We now study the performance of iGC under different client request levels. We run Cassandra Workload A with different numbers of YCSB client threads to evaluate the latency reduction due to iGC in comparison with ZGC under co-location.

Figure 10 shows the CDF of all requests of the tail latency when the number of client threads is set to 1, 2, 4, 8, 16, and 32 for iGC and ZGC. We can see that iGC obtains marginal improvement for the p90 and p95 tail latency under 1 and 2 threads. As the number of threads increases, the latency reduction due to iGC improves significantly. The reason is that there are contention among client threads, which increases the interference for CPU resources. With iGC, GC threads are assigned to different CPUs, which avoids the CPU contention with worker threads. As a result, iGC significantly alleviates the degradation of the tail latency. When there are 32 client threads, the latency reductions for p90, p95, and p99 due to iGC are 64%, 87%, and 83%, respectively. The results demonstrate that iGC has better scalability than ZGC.

To further demonstrate the scalability of iGC, we obtain the performance of iGC under different request per second (RPS) levels.

Table 5: The average latency (μs) of each operation.

Operations	Co W/ ZGC	Co W/ iGC
READ	1,169	860
UPDATE	1,010	682
INSERT	1,052	741
SCAN	31,124	35,700

Table 6: The Throughput by iGC upon G1 GC.

Workload	Cassandra (op/sec)			Batch
	A	B	D	(num/h)
Co w/ G1GC	1231	849	1240	187
Co w/ iGC	1783	1178	1711	183

We run Cassandra Workload A with different RPS, ranging from 500 to 5,000 with 500 as a step. Figure 11 shows the p95 tail latency due to iGC and ZGC. As the number of RPS increases, the p95 tail latency also increases. Compared to ZGC, iGC renders smaller but also more stable tail latency. It shows similar p95 latency from 500 to 3,500 RPS. On the other hand, the p95 latency due to ZGC deteriorates dramatically after the RPS reaches 2,500. The results again demonstrate that iGC has good scalability.

5.5 Microscopic View of Each Operation

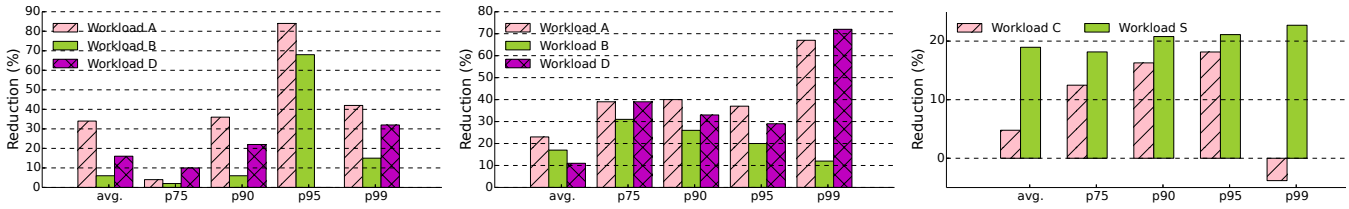
To evaluate the impact of iGC on each operation in LC services, we run four Cassandra operations (READ, UPDATE, INSERT, and SCAN) individually. Figure 12 shows the tail latency CDF of all requests of the four operations. Table 5 gives the average latency of each operation due to co-location with ZGC and iGC. iGC reduces the average latency of three operations READ, UPDATE, and INSERT by 28%, 32%, and 30%, respectively. For SCAN operation, the latency due to iGC is slightly longer than that due to ZGC. The reason is that a SCAN operation contains several sequential READ operations, which read data from the start to the end in a range. When Cassandra runs SCAN as the only operation, the allocation rate of new objects in Java heap is very high, which leads to consecutive GC. In such a case, iGC cannot adaptively trigger GC as GC keeps running. Thus, iGC leads to longer latency for SCAN. However, SCAN is not a common operation in NoSQL requests. There is a rare chance that SCAN operations execute one after another. As a result, iGC obtains the overall better performance for LC services.

5.6 G1 GC

In the previous experiments, we compare iGC with the default ZGC. Here, we experiment iGC upon G1 GC, another representative concurrent GC that is the default GC on JDK 9. We use Cassandra with workloads A, B, and D.

Table 6 shows the throughput of Cassandra workloads per second and that of batch jobs in one hour. iGC improves Cassandra throughput by 1.45X, 1.38X and 1.37X for workload A, B, and D, respectively. Its impact on the throughput of batch jobs is trivial. Note that the latency and throughput of LC services are the primary metrics for job co-location.

Figure 13 shows that compared to the default G1 GC under co-location, iGC reduces the p95 tail latency of Cassandra by 78%, 68%,



(a) Cassandra (b) HBase (c) Solr

Figure 8: Latency reduction of three LC services with different workloads in iGC.

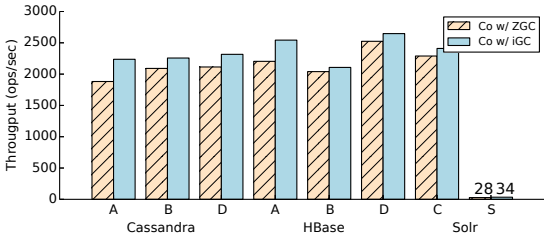


Figure 9: The throughput of co-located LC services.

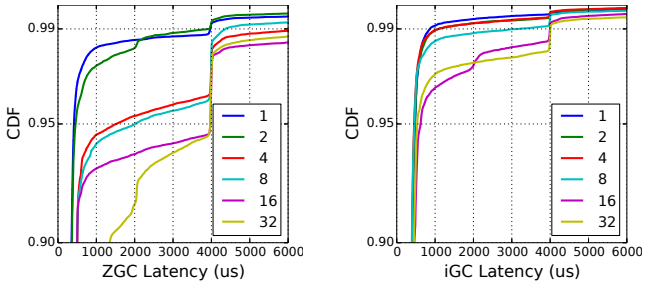


Figure 10: The tail latency CDF of all requests due to iGC and ZGC under different number of client threads.

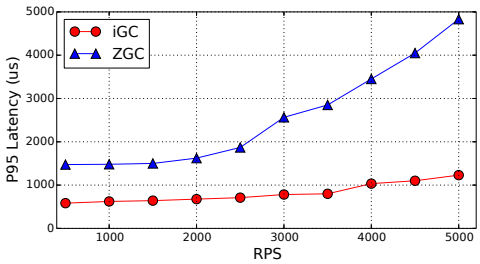


Figure 11: The p95 latency under different RPS.

and 72% for workload A, B, and D, respectively. We also notice that iGC reduces the p80 latency quite significantly, which is not the case in comparison of iGC with ZGC (as shown in Figure 5). There is a mechanism in G1 GC that shortens the pause time for each GC cycle. Because of that, GC is only triggered on part of the heap and G1 GC reallocates less memory in each GC cycle. As a result, GC

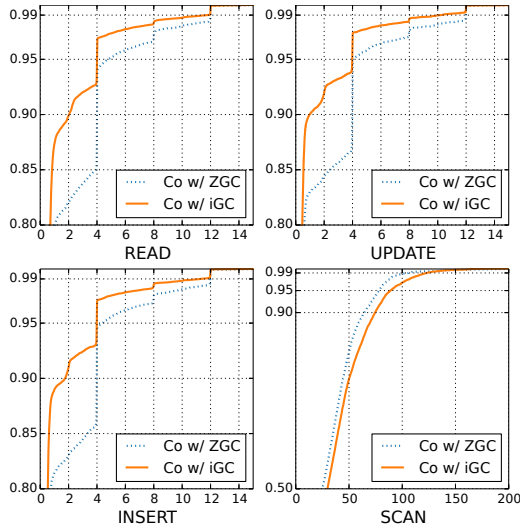


Figure 12: Microscopic view: The tail latency CDF of all requests (*ms*) for four Cassandra operations.

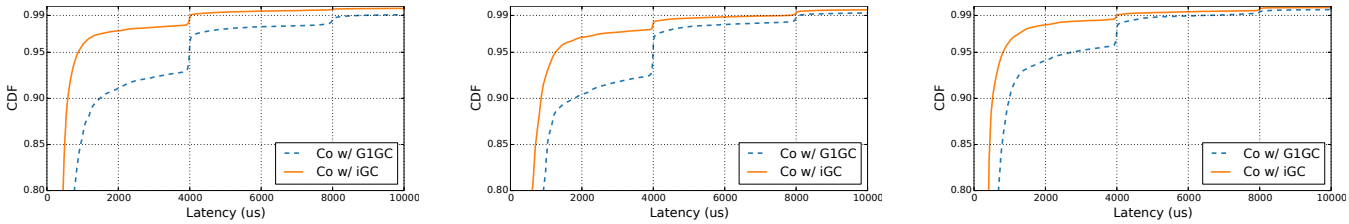
is triggered more frequently and the total GC time is prolonged. Longer GC time leads to more CPU resource contention and affects more operations of LC services. Thus, iGC is able to reduce the latency for low percentiles as well.

Overall, GC demonstrates significant improvement upon both ZGC and G1 GC.

5.7 Discussions

iGC Overhead. iGC separates GC threads from worker threads and place them onto different CPUs. Compared with ZGC or G1 GC, iGC uses one more CPU to run GC threads. Note that overhead of GC is limited at most one CPU in non-work-conserving manner regardless of the cluster size. As GC threads are not in the critical path of LC services, GC threads can share CPUs with threads of batch jobs. Additionally, GC threads only consume CPU resource when GC is triggered. In our evaluation, the CPU utilization of GC threads is around 30% of a CPU when GC is running. As long as iGC avoids allocation stall, its overhead is less than the capacity of one logical CPU.

iGC Downward Compatibility. Recently, JDK changes the naming convention so that there are many new versions (V11-V17). Currently, JDK 11 is used in production systems. Cassandra and



(a) Workload A (b) Workload B (c) Workload D

Figure 13: The tail latency CDF of all requests of three Cassandra workloads due to iGC upon G1 GC.

Hadoop support JDK 11 runtimes. G1 GC is the state-of-the-art mechanism for JDKs. Thus, iGC uses the newest-compatible Java version and supports downward compatibility.

6 RELATED WORK

LC service Optimization. Many studies focus on improving the QoS of LC services from the perspective of reducing tail latency [4, 20, 24, 46, 55]. For example, PerfIso [31] uses native CPU isolation to achieve CPU sharing between LC services and best-effort batch jobs. FlatStore [10] is a key-value store that uses logs in persistent memory for serving data requests. Caladan [24] proposes a new CPU scheduler in the Linux kerneal that improves QoS of LC services, especially the tail latency. Perséphone [20] is a novel kernel-bypass OS scheduler designed to minimize the tail latency for applications executing at the microsecond scale.

GC Optimization. There are studies on JVM optimization [5, 22, 28, 28, 38, 40, 43, 44, 52, 54]. For example, Facade [44] implements a compiler framework, which automatically stores data to native storage. ROLP [5] profiles object lifetime and uses the profiled information for efficient GC. Recently, Semeru [51] provides a distributed GC that offloads object tracing to memory servers so that tracing is performed closer to data in memory-disaggregated systems. Other studies focus on reducing the overhead of object serialization and caching intermediate data in memory. Deca [37] transforms application code to access native storage directly for performance improvement. Skyway [42] and FlashByte [54] propose to transfer objects from heap to native storage, which alleviates the serialization overhead. Gerenuk [41] is a compiler and runtime that aims to enable a JVM-based data-parallel system to achieve near native efficiency.

Resource Management in Multi-tenant Systems. Many studies [8–10, 23, 26, 32] focus on job scheduling and memory management in multi-tenant systems. For instance, HipsterCo [45] is a task scheduler for LC services running on multi-core systems. BIG-C [9] is a preemptive, low latency datacenter scheduling framework via lightweight containers. iTask [23] and UA killer [7] forcibly suspend tasks and performs node-level memory management to prioritize specific tasks. Hermes [46] offers fast memory allocation and proactive reclamation for reducing the latency of LC services under memory pressure. It targets applications written in C. iGC is an adaptive CPU allocation and thread placement approach that leverages on concurrent GC at runtime to tackle performance interference and reduce the tail latency of LC services. It targets Java applications that require GC.

Performance Interference and Isolation. Prior work has tackled performance interference in cloud systems in several ways. Some approaches disallow interactive services from sharing resources with other applications to avoid interference [36, 47, 50]. Some other approaches try to avoid co-scheduling applications that are likely to interfere with each other [18, 39]. The third category of research focus on eliminating interference altogether, by partitioning and isolating resources among co-scheduled services [19, 34–36, 46]. Parties [6] is a representative QoS-aware resource manager that enables a number of interactive LC services to share a physical node without QoS violations. iGC tackles the interference between GC threads and the worker threads of LC services, as well as the interference due to HT.

SMT Optimization. Several studies [21, 33, 53] focus on optimizing application performance running on SMT servers. Deng [21] proposes a symbiotic scheduling approach to solve the resource contention between virtual machines on SMT processors. vsmt-io [53] proposes to improve I/O performance and efficiency on SMT processors in virtualized clouds. iGC proposes to place different kinds of threads onto different CPUs while taking advantage of memory hierarchy.

7 CONCLUSION

iGC is motivated by the performance degradation due to CPU interference between concurrent GC threads and worker threads of LC services. Scheduling GC threads and worker threads onto separate processing units opens up the opportunity for performance improvement of LC services. Furthermore, iGC tackles HT interference by placing worker threads and batch threads onto different cores, but also takes advantage of HT capability by prioritizing the placement of worker threads and GC threads in the same cores for high cache utilization. iGC is designed and implemented as a ready-to-use middleware for JVM-based applications in Linux environments. Experimental results show that iGC significantly improves the performance of two state-of-the-art concurrent GC mechanisms (ZGC and G1 GC) for LC services and the throughput in multi-tenant systems.

In future, we plan to examine iGC performance on different types of processors and extend it for other OSes.

8 ACKNOWLEDGMENT

This research was supported in part by U.S. NSF grant CCF-1816850.

REFERENCES

[1] Apache. 2022. Apache Cassandra™. Retrieved Feb. 17, 2022 from https://cassandra.apache.org/

[2] Apache. 2022. Apache HBase™. Retrieved May 18, 2022 from https://hbase.apache.org/

[3] Apache. 2022. Apache Solr™. Retrieved May 12, 2022 from https://solr.apache.org/

[4] Daniel S Berger, Benjamin Berg, Timothy Zhu, Siddhartha Sen, and Mor Harchol-Balter. 2018. RobinHood: Tail Latency Aware Caching–Dynamic Reallocation from Cache-Rich to Cache-Poor. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18). Oct. 8–10, 2018, CARLSBAD, CA, USA*. USENIX Association, Carlsbad, CA, 195–212. https://doi.org/10.5555/3291168.3291183

[5] Rodrigo Bruno, Duarte Patricio, José Simão, Luis Veiga, and Paulo Ferreira. 2019. Runtime object lifetime profiler for latency sensitive big data applications. In *Proceedings of the Fourteenth EuroSys Conference 2019. March 25-28, 2019, Dresden Germany*. ACM, New York, NY, USA, 1–16. https://doi.org/10.1145/3302424.3303988

[6] Shuang Chen, Christina Delimitrou, and José F Martínez. 2019. Parties: Qos-aware resource partitioning for multiple interactive services. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. April 13-17, 2019, Providence, Rhode Island, USA*. ACM, New York, NY, USA, 107–120. https://doi.org/10.1145/3297858.3304005

[7] Wei Chen, Aidi Pi, Shaoqi Wang, and Xiaobo Zhou. 2019. Os-augmented oversubscription of opportunistic memory with a user-assisted oom killer. In *Proceedings of the 20th International Middleware Conference. Dec. 9-13, 2019, Davis CA USA*. ACM, New York, NY, USA, 28–40. https://doi.org/10.1145/3361525.3361534

[8] Wei Chen, Aidi Pi, Shaoqi Wang, and Xiaobo Zhou. 2019. Pufferfish: Container-driven elastic memory management for data-intensive applications. In *Proceedings of the ACM Symposium on Cloud Computing. Nov. 20-23, 2019, Santa Cruz CA USA*. ACM, New York, NY, USA, 259–271. https://doi.org/10.1145/3357223.3362730

[9] Wei Chen, Jia Rao, and Xiaobo Zhou. 2017. Preemptive, low latency datacenter scheduling via lightweight virtualization. In *2017 USENIX Annual Technical Conference (USENIX ATC 17). July 12–14, 2017, Santa Clara, CA*. USENIX Association, Santa Clara, CA, 251–263. https://doi.org/10.5555/3154690.3154714

[10] Youmin Chen, Youyou Lu, Fan Yang, Qing Wang, Yang Wang, and Jiwu Shu. 2020. Flatstore: An efficient log-structured key-value storage engine for persistent memory. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems. March 16-20, 2020, Lausanne, Switzerland*. ACM, New York, NY, USA, 1077–1091. https://doi.org/10.1145/3373376.3378515

[11] The Linux Kernel Development Community. 2012. Control Group v2. Retrieved Oct. 2015 from https://www.kernel.org/doc/html/v4.18/admin-guide/cgroup-v2.html

[12] Brian F Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM symposium on Cloud computing. June 10-11, 2010, Indianapolis Indiana USA*. ACM, New York, NY, USA, 143–154. https://doi.org/10.1145/1807128.1807152

[13] Oracle Corperation. 2016. Java Garbage Collection Stop The World Event. Retrieved May 20, 2016 from https://www.oracle.com/technetwork/tutorials/tutorials-1873457.html

[14] Intel Corporation. 2017. Intel® 64 and IA32 Architectures Performance Monitoring Events. Retrieved May 18, 2022 from https://software.intel.com/content/www/us/en/develop/download/intel-64-and-ia32-architectures-performance-monitoring-events.html

[15] Oracle Corporation. 2022. Java Management Extensions. Retrieved May 18, 2022 from https://docs.oracle.com/javase/8/docs/technotes/guides/management/agent.html

[16] Oracle Corporation. 2022. The Z Garbage Collector. Retrieved May 18, 2022 from https://openjdk.java.net/projects/zgc/

[17] Jeffrey Dean and Luiz André Barroso. 2013. The tail at scale. *Commun. ACM* 56, 2 (2013), 74–80. https://doi.org/10.1145/2408776.2408794

[18] Christina Delimitrou and Christos Kozyrakis. 2013. Paragon: QoS-aware scheduling for heterogeneous datacenters. *ACM SIGPLAN Notices* 48, 4 (2013), 77–88. https://doi.org/10.1145/2451116.2451125

[19] Christina Delimitrou and Christos Kozyrakis. 2017. Bolt: I know what you did last summer... in the cloud. *ACM SIGARCH Computer Architecture News* 45, 1 (2017), 599–613. https://doi.org/10.1145/3093337.3037703

[20] Henri Maxime Demoulin, Joshua Fried, Isaac Pedisich, Marios Kogias, Boon Thau Loo, Linh Thi Xuan Phan, and Irene Zhang. 2021. When Idling is Ideal: Optimizing Tail-Latency for Heavy-Tailed Datacenter Workloads with Perséphone. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles. Oct. 26-29, 2021, Virtual Event Germany*. ACM, New York, NY, USA, 621–637. https://doi.org/10.1145/3477132.3483571

[21] Kefeng Deng, Kaijun Ren, and Junqiang Song. 2012. Symbiotic scheduling for virtual machines on smt processors. In *2012 Second International Conference on Cloud and Green Computing. Nov. 1-3, 2012, Xiangtan China*. IEEE, 145–152. https://doi.org/10.1109/CGC.2012.54

[22] David Detlefs, Christine Flood, Steve Heller, and Tony Printezis. 2004. Garbage-first garbage collection. In *Proceedings of the 4th international symposium on Memory management. Oct. 24-25, 2004, Vancouver BC Canada*. ACM, New York, NY, USA, 37–48. https://doi.org/10.1145/1029873.1029879

[23] Lu Fang, Khanh Nguyen, Guoqing Xu, Brian Demsky, and Shan Lu. 2015. Interruptible tasks: Treating memory pressure as interrupts for highly scalable data-parallel programs. In *Proceedings of the 25th Symposium on Operating Systems Principles. Oct. 4-7, 2015, Monterey California*. ACM, New York, NY, USA, 394–409. https://doi.org/10.1145/2815400.2815407

[24] Joshua Fried, Zhenyuan Ruan, Amy Ousterhout, and Adam Belay. 2020. Caladan: Mitigating interference at microsecond timescales. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). Nov. 4-6, 2020, virtual*. USENIX Association, 281–297. https://doi.org/10.5555/3488766.3488782

[25] Yu Gan and Christina Delimitrou. 2018. The architectural implications of cloud microservices. *IEEE Computer Architecture Letters* 17, 2 (2018), 155–158. https://doi.org/10.1109/LCA.2018.2839189

[26] Eran Gilad, Edward Bortnikov, Anastasia Braginsky, Yonatan Gottesman, Eshcar Hillel, Idit Keidar, Nurit Moscovici, and Rana Shahout. 2020. EvenDB: optimizing key-value storage for spatial locality. In *Proceedings of the Fifteenth European Conference on Computer Systems. April 27-30, 2020, Heraklion Greece*. ACM, New York, NY, USA, 1–16. https://doi.org/10.1145/3342195.3387523

[27] Jambit GmbH. 2021. nice - change process priority. Retrieved Aug. 27, 2021 from https://man7.org/linux/man-pages/man2/nice.2.html

[28] Ionel Gog, Jana Giceva, Malte Schwarzkopf, Kapil Vaswani, Dimitrios Vytiniotis, Ganesan Ramalingam, Manuel Costa, Derek G Murray, Steven Hand, and Michael Isard. 2015. Broom: Sweeping out garbage collection from big data systems. In *15th Workshop on Hot Topics in Operating Systems (HotOS XV). May 18-20, 2015, Kartause Ittingen, Switzerland*. USENIX Association, USA. https://doi.org/10.5555/2831090.2831092

[29] Wessam M Hassanein, Layali K Rashid, and Moustafa A Hammad. 2008. Analyzing the effects of hyperthreading on the performance of data management systems. *International Journal of Parallel Programming* 36, 2 (2008), 206–225. https://doi.org/10.1007/s10766-007-0066-x

[30] Intel-bigdata. 2020. HiBench Suite. Retrieved Jan. 11, 2022 from https://github.com/Intel-bigdata/HiBench

[31] Călin Iorgulescu, Reza Azimi, Youngjin Kwon, Sameh Elnikety, Manoj Syamala, Vivek Narasayya, Herodotos Herodotou, Paulo Tomita, Alex Chen, Jack Zhang, et al. 2018. PerfIso: Performance Isolation for Commercial Latency-Sensitive Services. In *2018 USENIX Annual Technical Conference (USENIX ATC 18). July 11–13, 2018, Boston, MA, USA*. USENIX Association, USA, 519–532. https://doi.org/10.5555/3277355.3277406

[32] Călin Iorgulescu, Florin Dinu, Aunn Raza, Wajih Ul Hassan, and Willy Zwaenepoel. 2017. Don't cry over spilled records: Memory elasticity of data-parallel applications and its application to cluster scheduling. In *2017 USENIX Annual Technical Conference (USENIX ATC 17). July 12–14, 2017, Santa Clara, CA*. USENIX Association, Santa Clara, CA, 97–109. https://doi.org/10.5555/3154690.3154700

[33] Weiwei Jia, Jianchen Shan, Tsz On Li, Xiaowei Shang, Heming Cui, and Xiaoning Ding. 2020. vSMT-IO: Improving I/O Performance and Efficiency on SMT Processors in Virtualized Clouds. In *2020 USENIX Annual Technical Conference (USENIX ATC 20). July 15–17, 2020, virtual*. USENIX Association, USA, 449–463. https://doi.org/10.5555/3489146.3489176

[34] Harshad Kasture, Davide B Bartolini, Nathan Beckmann, and Daniel Sanchez. 2015. Rubik: Fast analytical power management for latency-critical systems. In *2015 48th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO). Dec. 5-12, 2015, Waikiki, HI, USA*. IEEE, 598–610. https://doi.org/10.1145/2830772.2830797

[35] Palden Lama, Shaoqi Wang, Xiaobo Zhou, and Dazhao Cheng. 2018. Performance isolation of data-intensive scale-out applications in a multi-tenant cloud. In *2018 IEEE International Parallel and Distributed Processing Symposium (IPDPS). May 21–25, 2018, Vancouver, British Columbia CANADA*. IEEE, 85–94. https://doi.org/10.1109/IPDPS.2018.00019

[36] David Lo, Liqun Cheng, Rama Govindaraju, Parthasarathy Ranganathan, and Christos Kozyrakis. 2015. Heracles: Improving resource efficiency at scale. In *Proceedings of the 42nd Annual International Symposium on Computer Architecture. June 13-17, 2015, Portland Oregon*. 450–462. https://doi.org/10.1145/2749469.2749475

[37] Lu Lu, Xuanhua Shi, Yongluan Zhou, Xiong Zhang, Hai Jin, Cheng Pei, Ligang He, and Yuanzhen Geng. 2016. Lifetime-Based Memory Management for Distributed Data Processing Systems. *Proc. VLDB Endow.* 9, 12 (2016). https://doi.org/10.14778/2994509.2994513

[38] Martin Maas, Tim Harris, Krste Asanović, and John Kubiatowicz. 2015. Trash day: Coordinating garbage collection in distributed systems. In *15th Workshop on Hot Topics in Operating Systems (HotOS XV). May 18-20, 2015, Kartause Ittingen, Switzerland*. USENIX Association, USA. https://doi.org/10.5555/2831090.2831091

[39] Jason Mars and Lingjia Tang. 2013. Whare-map: Heterogeneity in" homogeneous" warehouse-scale computers. In *Proceedings of the 40th Annual International Symposium on Computer Architecture. June 23-27, 2013, Tel-Aviv Israel*, Vol. 41. ACM, New York, NY, USA, 619–630. https://doi.org/10.1145/2508148.2485975

[40] Kris Mok. 2011. JVM@Taobao. Retrieved Oct. 24, 2011 from https://www.slideshare.net/RednaxelaFX/jvm-taobao

[41] Christian Navasca, Cheng Cai, Khanh Nguyen, Brian Demsky, Shan Lu, Miryung Kim, and Guoqing Harry Xu. 2019. Gerenuk: thin computation over big native data using speculative program transformation. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles. Oct. 27-30, 2019, Huntsville Ontario Canada*. ACM, New York, NY, USA, 538–553. https://doi.org/10.1145/3341301.3359643

[42] Khanh Nguyen, Lu Fang, Christian Navasca, Guoqing Xu, Brian Demsky, and Shan Lu. 2018. Skyway: Connecting managed heaps in distributed big data systems. *ACM SIGPLAN Notices* 53, 2 (2018), 56–69. https://doi.org/10.1145/3173162.3173200

[43] Khanh Nguyen, Lu Fang, Guoqing Xu, Brian Demsky, Shan Lu, Sanazsadat Alamian, and Onur Mutlu. 2016. Yak: A High-Performance Big-Data-Friendly Garbage Collector. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16). Nov. 2–4, 2016, SAVANNAH, GA, USA*. USENIX Association, USA, 349–365. https://doi.org/10.5555/3026877.3026905

[44] Khanh Nguyen, Kai Wang, Yingyi Bu, Lu Fang, Jianfei Hu, and Guoqing Xu. 2015. Facade: A compiler and runtime for (almost) object-bounded big data applications. *ACM SIGARCH Computer Architecture News* 43, 1 (2015), 675–690. https://doi.org/10.1145/2775054.2694345

[45] Rajiv Nishtala, Paul Carpenter, Vinicius Petrucci, and Xavier Martorell. 2017. Hipster: Hybrid task manager for latency-critical cloud workloads. In *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA). Feb. 4-8 2017, Austin, Texas*. IEEE, 409–420. https://doi.org/10.1109/HPCA.2017.13

[46] Aidi Pi, Junxian Zhao, Shaoqi Wang, and Xiaobo Zhou. 2021. Memory at your service: fast memory allocation for latency-critical services. In *Proceedings of the 22nd International Middleware Conference. Dec. 6-10, 2021, Québec city Canada*. ACM, New York, NY, USA, 185–197. https://doi.org/10.1145/3464298.3493394

[47] Malte Schwarzkopf, Andy Konwinski, Michael Abd-El-Malek, and John Wilkes. 2013. Omega: Flexible, Scalable Schedulers for Large Compute Clusters. In *Proceedings of the 8th ACM European Conference on Computer Systems. April 15-17, 2013, Prague Czech Republic*. ACM, New York, NY, USA, 351–364. https://doi.org/10.1145/2465351.2465386

[48] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. 2010. The hadoop distributed file system. In *2010 IEEE 26th symposium on mass storage systems and technologies (MSST). May 3-7, 2010, Lake Tahoe, Nevada, USA*. IEEE, 1–10. https://doi.org/10.1109/MSST.2010.5496972

[49] Vinod Kumar Vavilapalli, Arun C Murthy, Chris Douglas, Sharad Agarwal, Mahadev Konar, Robert Evans, Thomas Graves, Jason Lowe, Hitesh Shah, Siddharth Seth, et al. 2013. Apache hadoop yarn: Yet another resource negotiator. In *Proceedings of the 4th annual Symposium on Cloud Computing. Oct. 1-3, 2013, Santa Clara California*. ACM, New York, NY, USA, 1–16. https://doi.org/10.1145/2523616.2523633

[50] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-Scale Cluster Management at Google with Borg. In *Proceedings of the Tenth European Conference on Computer Systems. April 21-24, 2015, Bordeaux France*. Association for Computing Machinery, New York, NY, USA, 1–17. https://doi.org/10.1145/2741948.2741964

[51] Chenxi Wang, Haoran Ma, Shi Liu, Yuanqi Li, Zhenyuan Ruan, Khanh Nguyen, Michael D Bond, Ravi Netravali, Miryung Kim, and Guoqing Harry Xu. 2020. Semeru: A Memory-Disaggregated Managed Runtime. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20). Nov. 4-6, 2020, virtual*. USENIX Association, USA, 261–280. https://doi.org/10.5555/3488766.3488781

[52] Jingjing Wang and Magdalena Balazinska. 2017. Elastic memory management for cloud data analytics. In *2017 USENIX Annual Technical Conference (USENIX ATC 17). July 12–14, 2017, Santa Clara, CA*. USENIX Association, USA, 745–758. https://doi.org/10.5555/3154690.3154760

[53] Xi Yang, Stephen M Blackburn, and Kathryn S McKinley. 2016. Elfen Scheduling: Fine-GrainPrincipled Borrowing from Latency-Critical Workloads Using Simultaneous Multithreading. In *2016 USENIX Annual Technical Conference (USENIX ATC 16). July 12-14, 2017, Santa Clara, CA*. USENIX Association, USA, 309–322. https://doi.org/10.5555/3026959.3026988

[54] Junxian Zhao, Aidi Pi, Shaoqi Wang, and Xiaobo Zhou. 2021. Flashbyte: Improving memory efficiency with lightweight native storage. In *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid). May 10-13, 2021, Melbourne, Australia*. IEEE, 61–70. https://doi.org/10.1109/CCGrid51090.2021.00016

[55] Timothy Zhu, Michael A Kozuch, and Mor Harchol-Balter. 2017. Workloadcompactor: Reducing datacenter cost while providing tail latency slo guarantees. In *Proceedings of the 2017 Symposium on Cloud Computing. Sept. 24-27, 2017, Santa Clara California*. ACM, New York, NY, USA, 598–610. https://doi.org/10.1145/3127479.3132245